If you specify your own `select`, be sure to include the primary key and foreign key columns of the associated model. If you do not, Rails will throw an error.

4.3.3.10 `distinct`

Use the `distinct` method to keep the collection free of duplicates. This is mostly useful together with the `:through` option.

```
class Person < ActiveRecord::Base
  has_many :readings
  has_many :articles, through: :readings
end

person = Person.create(name: 'John')
article   = Article.create(name: 'a1')
person.articles << article
person.articles << article
person.articles.inspect # => [#<Article id: 5, name: "a1">, #<Article
id: 5, name: "a1">]
Reading.all.inspect  # => [#<Reading id: 12, person_id: 5, article_id:
5>, #<Reading id: 13, person_id: 5, article_id: 5>]
```

In the above case there are two readings and `person.articles` brings out both of them even though these records are pointing to the same article.

Now let's set `distinct`:

```
class Person
  has_many :readings
  has_many :articles, -> { distinct }, through: :readings
end

person = Person.create(name: 'Honda')
article   = Article.create(name: 'a1')
person.articles << article
person.articles << article
person.articles.inspect # => [#<Article id: 7, name: "a1">]
Reading.all.inspect  # => [#<Reading id: 16, person_id: 7, article_id:
7>, #<Reading id: 17, person_id: 7, article_id: 7>]
```

In the above case there are still two readings. However `person.articles` shows only one article because the collection loads only unique records.

If you want to make sure that, upon insertion, all of the records in the persisted association are distinct (so that you can be sure that when you inspect the association that you will never find duplicate records), you should add a unique index on the table itself. For example, if you have a table named `person_articles` and you want to make sure all the articles are unique, you could add the following in a migration:

```
add_index :person_articles, :article, unique: true
```

Note that checking for uniqueness using something like `include?` is subject to race conditions. Do not attempt to use `include?` to enforce distinctness in an association. For instance, using the article example from above, the following code would be racy because multiple users could be attempting this at the same time:

```
person.articles << article unless person.articles.include?(article)
```

4.3.4 When are Objects Saved?

When you assign an object to a `has_many` association, that object is automatically saved (in order to update its foreign key). If you assign multiple objects in one statement, then they are all saved.

If any of these saves fails due to validation errors, then the assignment statement returns `false` and the assignment itself is cancelled.

If the parent object (the one declaring the `has_many` association) is unsaved (that is, `new_record?`returns `true`) then the child objects are not saved when they are added. All unsaved members of the association will automatically be saved when the parent is saved.